

Министерство науки и образования Российской Федерации  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

## **ОТЧЕТ**

по лабораторной работе №2  
по курсу «Логика и основы  
алгоритмизации в инженерных задачах»  
на тему «Оценка времени выполнения программ»

Выполнил:

студент группы 25BBB1  
Цеплов Е.В.  
Разживалов Н.А.

Приняли:

к.т.н., доцент Юрова О.В.,  
к.т.н., доцент Деев М.В.

Пенза 2026

## Название

Оценка времени выполнения программ

## Цель работы

Оценка времени выполнения программ

## Лабораторное задание

Дана программа, вычисляющая произведение двух матриц.

Задание 1:

1. Вычислить порядок сложности программы ( $O$ -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Даны реализации алгоритмов сортировки Шелла и быстрой сортировки.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

## Ход работы

Задание 1:

1. В программе используется три вложенных цикла

```

for (i = 0; i < 200; i++)
{
    for (j = 0; j < 200; j++)
    {
        elem_c = 0;
        for (r = 0; r < 200; r++)
        {
            elem_c = elem_c + a[i][r] * b[r][j];
            c[i][j] = elem_c;
        }
    }
}

```

Сложность  $O(n^3)$ .

- Для оценки времени выполнения программы модифицировал исходный код программы добавив подсчет времени а так же еще один цикл для последовательной проверки, а так же вывод количества секунд.

```

int sizes[] = { 100, 200, 400, 1000, 2000, 4000, 10000 };
int count = sizeof(sizes) / sizeof(sizes[0]);

srand((unsigned)time(NULL));

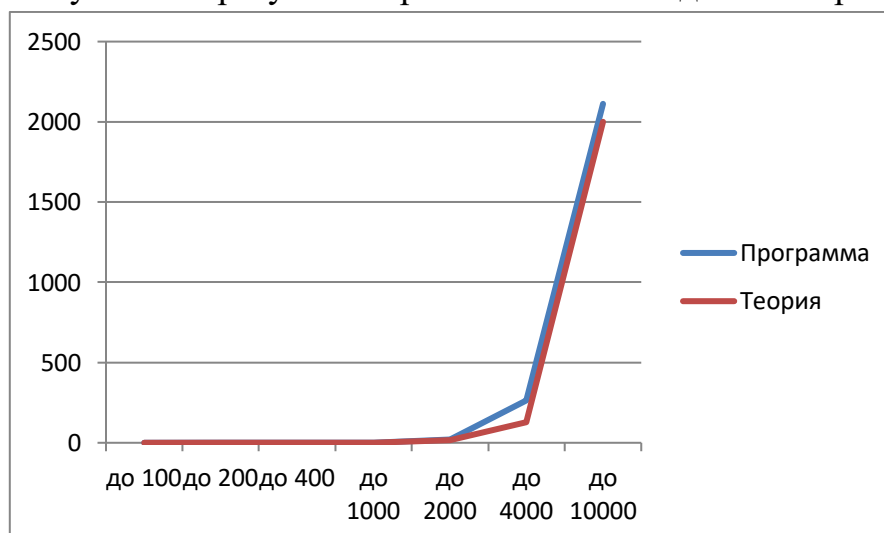
for (int s = 0; s < count; s++)

end = clock();

printf("Matric: %d x %d\n", n, n);
printf("Time: %.6f sec\n", (double)(end - start) / CLOCKS_PER_SEC);

```

- Полученный результат практически совпадает с теорией.



| 1 |          | Программ  | Теория |
|---|----------|-----------|--------|
| 2 | до 100   | 0,002     | 0,002  |
| 3 | до 200   | 0,019     | 0,016  |
| 4 | до 400   | 0,117     | 0,128  |
| 5 | до 1000  | 2,063     | 2      |
| 6 | до 2000  | 21,289    | 16     |
| 7 | до 4000  | 264,736   | 128    |
| 8 | до 10000 | 2112,0000 | 2000   |

## Задание 2:

Когда 10000 элементов:

| Тип массива   | Shell     | Qs        | Qsort     |
|---------------|-----------|-----------|-----------|
| Случайный     | 0.372 сек | 0.008сек  | 0.044 сек |
| Возрастающий  | 0.001 сек | 0.002 сек | 0.033 сек |
| Убывающий     | 0.776 сек | 0.002 сек | 0.034 сек |
| Возр. и Убыв. | 0.391 сек | 0.008 сек | 0. 13 сек |

Когда 50000 элементов:

| Тип массива   | Shell      | Qs        | Qsort      |
|---------------|------------|-----------|------------|
| Случайный     | 9.46 сек   | 0.043 сек | 0.215 сек  |
| Возрастающий  | 0.004 сек  | 0.009 сек | 0.188 сек  |
| Убывающий     | 20.671 сек | 0.01 сек  | 0.206 сек  |
| Возр. и Убыв. | 10.388 сек | 0.04 сек  | 0. 814 сек |

Для выполнения второго задания потребовалось добавить переменную temp для записи в него массива не меняя исходный. А так же 4 цикла для записи в массив items.

```
srand((unsigned)time(NULL));
for (int i = 0; i < count;i++) {
    items[i] = rand() % 10000;
}
for (int i = 0; i < count; i++){
    temp[i] = items[i];
}
```

## Результат работы программы

Задание 1:

```
Matric: 100 x 100
Time: 0.002000 sec

Matric: 200 x 200
Time: 0.016000 sec

Matric: 400 x 400
Time: 0.107000 sec

Matric: 1000 x 1000
Time: 2.151000 sec

Matric: 2000 x 2000
Time: 21.332000 sec

Matric: 4000 x 4000
Time: 237.468000 sec
```

## Задание 2:

```
Var in 100000
Random massiv
Shell_Time 0.372000 sec
Qs_Time 0.008000 sec
Qsort_Time 0.044000 sec
Upper Massiv
Shell_Time 0.001000 sec
Qs_Time 0.002000 sec
Qsort_Time 0.033000 sec
Down Massiv
Shell_Time 0.776000 sec
Qs_Time 0.002000 sec
Qsort_Time 0.034000 sec
UPandDown Massiv
Shell_Time 0.391000 sec
Qs_Time 0.008000 sec
Qsort_Time 0.130000 sec
```

```
Var in 500000
Random massiv
Shell_Time 9.460000 sec
Qs_Time 0.043000 sec
Qsort_Time 0.215000 sec
Upper Massiv
Shell_Time 0.004000 sec
Qs_Time 0.009000 sec
Qsort_Time 0.188000 sec
Down Massiv
Shell_Time 20.671000 sec
Qs_Time 0.010000 sec
Qsort_Time 0.206000 sec
UPandDown Massiv
Shell_Time 10.388000 sec
Qs_Time 0.040000 sec
Qsort_Time 0.814000 sec
```

## Вывод

В ходе лабораторной работы было измерено время выполнения алгоритмов умножения матриц и сортировки массивов. Установлено, что умножение матриц имеет сложность  $O(n^3)$ , а время сортировки зависит от используемого алгоритма и типа исходного массива. Полученные результаты позволяют сравнить практическое время работы алгоритмов с их теоретической сложностью.

Лидер — Qs: Самый быстрый и стабильный алгоритм на всех типах данных и объемах. Qsort — надежная альтернатива: Показывает предсказуемое время работы, но стабильно уступает Qs в скорости. Shell — худший выбор для больших данных: Катастрофически замедляется на случайных и убывающих

массивах (до 20 сек на 50 000 элементов). Его использование оправдано только для небольших или уже отсортированных массивов.